

Game 24 Puzzle

1. Project Overview

This project is a puzzle game based on the classic 24 Game. Players are given a set of numbers (typically four) and must combine them using mathematical operations to reach the target number, which is usually 24.

Players can use basic operations such as addition, subtraction, multiplication, and division. In harder modes, additional operations like power and square root may also be used.

The game includes a Hint mode to help players when they are stuck. There is no final level, so players can continue solving puzzles and increasing their total score. A Speed mode is also included, where players must solve puzzles within a limited time.

2. Project Review

The project is inspired by the traditional 24 Game, which is commonly used to practice arithmetic and logical thinking.

However, many existing versions only focus on solving puzzles. This project improves the original idea by adding more features and different game modes.

- Multiple Game Modes (Classic, Hard, Advanced, Speed, Practice) – Players can choose different difficulty levels.
- Advanced Mode – Players are given five numbers and a random target, increasing the difficulty.
- Score Tracking – The system records player scores.
- Statistical Data Collection and Analysis – Gameplay data can be analyzed and displayed using charts.

These features make the game more than just a puzzle game, but also a tool to analyze player performance.

3. Programming Development

3.1 Game Concept

In each round, the game generates a set of numbers and a target value. Players must use each number exactly once to create an expression that equals the target.

The main goal is to solve the puzzle correctly and get a higher score. Different game modes provide different levels of difficulty.

In Advanced mode :

- Number ranges are higher
- Target values are more flexible
- Advanced operators (^) and (√) are enabled

This mode is designed for players who want more challenges.

3.2 Object-Oriented Programming Implementation

The game is developed using Object-Oriented Programming (OOP). Different classes are used to organize the program and separate each responsibility clearly.

1. **User Class** - This class represents a user account in the game
Attributes : username, password
Methods : login(), logout()
2. **GameSession Class** - This class manages a single game session (handled within the main game logic)
Attributes : mode, timeUsed, attempts
Methods : start_game(), end_game()
3. **Puzzle Class** - This class generates puzzle numbers and the target value.
Attributes : numbers, target
Methods : generate(), validate()

4. **MathEngine Class** - This class handles calculations and validates expressions.
Attributes: operators, allowedFunctions
Methods: evaluate_expression(), check_solution()
5. **ScoreCalculator Class** - This class calculates the player's score.
Attributes: baseScore, timeBonus
Methods: calculate_score(), apply_bonus()
6. **Stats Class** - This class records player statistics.
Attributes: gamesPlayed, gamesWon, totalScore
Methods: record_game(), calculate_winrate()
7. **ExpressionBuilder Class** - This class builds the player's mathematical expression.
Attributes: expression, usedNumbers
Methods: add_number(), add_operator()
8. **App Class** - This class controls the overall game flow and interface.
Attributes: currentUser, gameState
Methods: start_game(), switch_screen()

3.3 Algorithms Involved

Several algorithms are used in the system:

Puzzle Generation – The system generates four random numbers and checks if they can form the target value before using them. It does this by trying all possible arrangements of the numbers, different operators (+, −, ×, ÷), and calculation orders. If at least one combination works, the set is used; otherwise, new numbers are generated.

Brute-force Solver – The solver tries all possible ways to arrange the given numbers and combine them with operators (+, −, ×, ÷), including different calculation orders. Each expression is evaluated using the MathEngine, and if any result equals the target value, the puzzle is considered solvable.

Expression Validation – The program checks whether the player's mathematical expression is valid and matches the target value.

Score Calculation – The score is calculated based on multiple factors such as base score, time bonus, and hint penalty.

Statistical Logging – The system records gameplay data at the game level (per session) for analysis.

4. Statistical Data (Prop Stats)

4.1 Data Features

The system records gameplay data for every completed session.

Recorded data includes:

username – The name of the player who played the game.

score – The total score earned during that session.

mode – The selected game mode (Classic, Hard, Advanced, Speed, Practice)

won – The result of the game session (win or loss).

time_used – The total time taken to complete the game.

attempts – The number of attempts made by the player.

hints_used – The number of hints used during the game.

4.2 Data Recording Method

All gameplay data is stored in a CSV file: `game_log.csv`

Each row represents one completed game session.

Example structure:

username, score, mode, won, time_used, attempts, hints_used

The data collection process is manageable. Each game takes about 30–60 seconds, so collecting 100 records can be done quickly. The data can also be gathered from multiple users and sessions, so it does not rely on a single player.

The game includes different modes (such as Classic, Hard, Advanced, Speed, and Practice), which helps create more varied data without making gameplay repetitive. All data is recorded automatically after each game, so it does not interrupt normal gameplay.

4.3 Data Analysis Report

The recorded data will be visualized using matplotlib. The following graphs will be generated automatically from the CSV file.

Examples of analysis include:

- attempts – represented by a histogram showing the distribution of attempts per session.
- time_used – represented by a line chart showing trends of solving time across multiple sessions.
- attempts vs time_used – represented by a scatter plot showing the relationship between problem-solving effort and time taken.
- won – represented by a pie chart showing the proportion of wins and losses.
- hints_used – represented by a bar chart comparing hint usage across sessions.

Visualization Requirements

Data Features

	Why is it good to have this data? What can it be used for?	How will you obtain 100 values of this feature data?	Which variable (and which class) will you collect this from?	How will you display this feature data?
attempts	Measures problem-solving difficulty and player effort. Used to analyze how many tries players need per game.	One value recorded per completed game session. Play multiple sessions to collect 100 values.	attempts in GameSession / PlayerStats	Histogram Statistic Table
time_used	Measures player speed and efficiency. Used to track improvement over time.	One value recorded per session.	time_used in GameSession	Line chart
attempts vs time_used	Shows the relationship between effort and solving time. Helps analyze player behavior and strategy complexity.	Derived from combining attempts and time_used across sessions.	attempts & time_used in GameSession	Scatter plot
won	Measures overall success rate. Used to evaluate player performance.	One value recorded per session (win or loss).	won in GameSession / Stats	Pie chart Statistic Table
hints_used	Measures reliance on hints. Used to analyze player difficulty and support usage.	One value recorded per session.	hints_used in GameSession	Bar chart Statistic Table

Statistic Table

Feature	Statistical Value
attempts	mean, median, minimum, maximum, standard deviation
hints_used	mean, median, minimum, maximum, standard deviation
won	count, percentage

Graph Plan

Feature Name	Graph Objective	Graph Type	X-axis	Y-axis
attempts	To analyze how many attempts players typically need and measure puzzle difficulty	Histogram	Number of attempts	Frequency
time_used	To observe trends in solving time and track player improvement	Line chart	Game sessions (index)	Time used (seconds)
attempts vs time_used	To examine the relationship between effort and solving time (correlation)	Scatter plot	Number of attempts	Time used (seconds)
won	To show the proportion of wins and losses	Pie chart	Result (Win/Loss)	Percentage / Count
hints_used	To compare hint usage across sessions	Bar chart	Game sessions	Number of hints used

5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Learn and implement basic Pygame interface and UI design
3	10	Create a simple GUI for gameplay
4	11	Implement puzzle generation system
5	12	Implement stats recording using CSV
6	13	Testing the game and fixing bugs
7	14	Submission week (Draft)

6. Document version

Version: 2.0

Date: 26 March 2026

Editor: Napakhet Namsrioun

Date	Name	Feedback
17/03	Peraya	Major Changes Needed - Section 4.1 Statistical Data Viability: As your TA, I am flagging a significant logical gap in your data collection strategy. You listed Score, Mode, and Result as features. These are recorded once per game session. To reach the 100-record requirement per feature, a user would have to finish 100 full games. This is considered tiresome and inefficient for a Year Project. - Section 3.3 Algorithms: You mentioned the system ensures at least one valid solution. However, generating 4 random numbers and <i>then</i> checking for a solution is computationally heavy if done poorly. You need to detail the algorithm used to verify that a set of 4 numbers is actually solvable before presenting it to the player. Without this detail, the Puzzle Generation is just a vague idea, not a technical plan.

		Minor Changes Needed - Section 3.2 OOP Implementation: You have 8 classes, which is good and exceeds the minimum requirement. The relationship between User and Player is slightly redundant. The player usually inherits from a User base class or acts as a component within a session. - Section 4.3 Data Analysis Report: You mentioned Average score. To make this a Year Project level analysis, suggest analyzing the correlation between Number of Operators used and Solving Time. This shows you are using data to understand human behavior, not just counting wins.
18/03	Amornrit	Adding a few more features would be nice for your game identity (optional but highly encouraged). Expand the puzzle generation; how do you know that the set of numbers is solvable? Statistical Data: Collecting proposed data features can be time-consuming. So think about it carefully. Moreover, the Statistical Analysis section needs a detailed explanation.
30/03	Peraya	Overall, good work. - You have a Graph Plan and a Feature Table, but you are missing the Table of Statistical Values (showing Mean, Median, Max, Min, etc.). The rubric requires at least 1 table specifically for showing statistical values.